

Credit-Card Fraud Detection

Yarden Greenhut

Data Science in Cyber - Final Project

July 2026

Abstract

This project critically evaluates a popular open-source credit-card fraud detection tutorial ([anshpandey96/Credit-Card-Fraud-Detection](#)). We faithfully reproduce the author's pipeline on the Kaggle ULB dataset and test its headline claims (Logistic Regression: ~99.8% accuracy, 100% recall, ~94% precision). We find the code is largely sound but the claims are not supported by evidence: accuracy is a misleading metric under a 0.172% fraud rate, and the advertised precision/recall are not reproducible. We re-evaluate with imbalance-appropriate metrics (PR-AUC, MCC, F_2) and compare five models.

Contents

1	Evaluation Metrics - definitions	3
2	Summary of the Source	3
3	Critical Evaluation	5
4	Feature Engineering Analysis	6
5	Reproducibility Analysis	7
6	Experimental Results	7
7	Error Analysis	8
8	Conclusions	9

1 Evaluation Metrics - definitions

We first give the mathematical definition for the metrics that will be used in the report. Let TP (True Positive), TN (True Negative), FP (False Positive), FN (False Negative) denote the confusion matrix counts.

- **Precision** = $\frac{TP}{TP + FP}$.
- **Recall** = $\frac{TP}{TP + FN}$.
- $F_\beta = (1 + \beta^2) \frac{\text{precision} \cdot \text{recall}}{\beta^2 \text{precision} + \text{recall}}$. We emphasize F_2 ($\beta = 2$) because a missed fraud is costlier than a false alarm here.
- **MCC** = $\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \in [-1, 1]$.
- **PR-AUC** (Average Precision): area under the precision-recall curve.
- **ROC-AUC**: reported but *critiqued* as optimistic here.

We **exclude accuracy** as a headline metric and justify why (the Dummy baseline).

2 Summary of the Source

Problem addressed. The source addresses the problem of detecting fraudulent credit-card transactions using supervised machine-learning classification techniques. Credit-card fraud detection is a binary classification problem in which each transaction must be assigned to one of two classes: legitimate transactions ($Class = 0$) or fraudulent transactions ($Class = 1$).

The main difficulty of this problem is that fraudulent transactions represent only a very small fraction of all transactions, making it a highly imbalanced classification problem and an anomaly-detection task.

In highly imbalanced datasets, a conventional classifier may achieve very high accuracy simply by predicting the majority class. For example, a model that predicts every transaction as legitimate would still achieve approximately 99.8% accuracy on our dataset (explained in details in next parts of the section) while failing to identify any fraud.

Therefore, performance must be evaluated using metrics that measure the ability to detect the minority fraud class.

Why it matters. Credit-card fraud represents a significant financial and operational challenge for banks, payment providers, merchants, and customers. Fraudulent transactions result in direct money losses, chargeback costs, investigation expenses, and damage to customer trust. Because of the imbalance between legitimate and fraudulent transactions, the costs of classification errors are not equal.

A **False Negative (FN)** occurs when a fraudulent transaction is incorrectly classified as legitimate. This error is particularly costly because the fraudulent payment may be completed, resulting in direct financial loss. A **False Positive (FP)** occurs when a legitimate transaction

is incorrectly classified as fraud. Although less financially damaging, false positives negatively affect customer experience by blocking valid payments, creating additional verification steps, and increasing customer-support costs.

Consequently, a practical fraud-detection system must balance fraud detection capability with customer convenience. Metrics such as recall, precision, and the F_1 -score are more appropriate than accuracy alone because they explicitly measure this trade-off.

Dataset. The source uses the publicly available Kaggle *Credit Card Fraud Detection* dataset created by the Machine Learning Group of Université Libre de Bruxelles (ULB). The dataset contains 284,807 credit-card transactions collected over two days, of which only 492 transactions are fraudulent. This means that fraudulent transactions account for only approximately 0.172% of the total dataset.

The dataset contains numerical transaction attributes including transaction time, transaction amount, anonymized variables generated through Principal Component Analysis (PCA), and the binary target label. The original customer and transaction information was transformed using PCA to protect confidentiality while preserving useful statistical patterns for machine-learning applications.

The features include **Time**, representing the number of seconds elapsed since the first recorded transaction, **Amount**, representing the transaction value in euros, **Class**, representing the target label, and **V1–V28**, which are anonymized PCA-transformed variables derived from the original confidential features.

See Table 1 for the dataset structure and statistics.

Proposed solution (the source). The source implements a machine-learning pipeline for fraud detection using data preprocessing, class balancing, model training, and performance evaluation. The main steps are:

1. **Data preprocessing:** The numerical transaction features are prepared for machine learning. Feature scaling is applied so that variables with different numerical ranges do not dominate the learning process.
2. **Handling class imbalance:** Since fraudulent transactions represent only 0.172% of the dataset, the source applies the **Synthetic Minority Oversampling Technique (SMOTE)**. SMOTE creates synthetic examples of fraudulent transactions by interpolating between existing minority-class samples. This increases the representation of fraud cases during training and helps the classifier learn fraud-related patterns.
3. **Model training:** Two supervised learning algorithms are trained: **Logistic Regression** and **Random Forest**. Logistic Regression provides a simple linear decision boundary and serves as a baseline classification approach, whereas Random Forest is an ensemble model capable of capturing nonlinear patterns by combining multiple decision trees.
4. **Evaluation:** The trained models are evaluated using
 - **Accuracy:** The proportion of correctly classified transactions. While useful as a general measure, it can be misleading for extremely imbalanced datasets as ours.
 - **Precision:** The proportion of transactions predicted as fraudulent that are actually fraudulent. High precision reduces unnecessary blocking of legitimate customers.
 - **Recall:** The proportion of actual fraudulent transactions that are successfully detected. High recall is important because missed fraud cases directly translate into financial losses.

- **F_1 -score:** The harmonic mean of precision and recall, providing a single metric that balances fraud detection performance and false-alarm reduction.

According to the source, the models achieve approximately 99.8% accuracy, nearly 100% recall, and around 94% precision after balancing the dataset.

Although the reported results indicate strong performance, accuracy should be interpreted carefully because it is strongly influenced by the majority legitimate class. In fraud detection, the ability to identify fraudulent transactions (recall) while limiting false alarms (precision) is generally more important than maximizing accuracy.

Table 1: Dataset schema and key statistics (raw dataset before preprocessing).

Column(s)	Type	Meaning	Notes
Time	Numeric	Seconds since first transaction	0–172,792 s ($\approx$ 48 h)
V1–V28	Numeric ($\times 28$)	PCA anonymized components	$\mu \approx 0$, $\sigma \approx 1$
Amount	Numeric	Transaction value (€)	Median $\approx$ 22, max $\approx$ 25,691
Class	Binary {0, 1}	Fraud label (1 = fraud)	492 frauds / 284,807 total

3 Critical Evaluation

Main claims of the author (from the repo README).

- Best model = Logistic Regression with $\sim 99.8\%$ accuracy, 100% recall, $\sim 94\%$ precision.
- SMOTE resolves the class-imbalance problem.
- The reported metrics demonstrate an effective fraud detector.

Are the claims supported by the evidence? No, the headline numbers are not. Importantly, the critique is not that the code is buggy. The source correctly splits before scaling, fits the scaler on the training set only, applies SMOTE to the training set only, and selects the best model by F_1 (not accuracy). Those are genuine strengths and we credit them. The problems are in the **claims** and the **metric choices** (See table 2):

1. **The accuracy fallacy.** With only 0.172% positives, a `DummyClassifier` that always predicts “legitimate” achieves $\approx 99.83\%$ accuracy while catching **zero** frauds. The README leads with “ $\sim 99.8\%$ accuracy”, so its headline metric is essentially uninformative. (*Evidence in the notebook*)
2. **The headline performance is not reproducible.** Re-running the author’s *exact* pipeline (same split, scaling, SMOTE, models) on the real test set does **not** reproduce “100% recall / 94% precision” for Logistic Regression. Observed: recall $\sim 87.4\%$ and precision $\sim 5.3\%$.

The precision claim in particular collapses, because Logistic Regression on SMOTE-balanced data produces many false positives. A single optimistic run might have been reported as a general result. (*Evidence: notebook part 3, claim-vs-observed table.*)

3. Inappropriate and missing evaluation metrics.

The source reports only accuracy/precision/recall/ F_1 . Under extreme imbalance the domain appropriate metrics are **PR-AUC** (Average Precision), **MCC**, and F_β ($\beta = 2$) (ROC-AUC is over optimistic because true negatives dominate).

On these, our models score PR-AUC ~ 0.824 / MCC ~ 0.83 / $F_2\sim 0.8$, which is solid, but far from “near-perfect”. (*Evidence: notebook parts 4 and 6*)

Further analysis. We intentionally test the common SMOTE-before-split leakage mistake and measure how much it inflates performance (recall $\sim 87.4\% \rightarrow \sim 97.4\%$). The results show that the author *avoided* this error. The high reported performance is therefore unlikely caused from data leakage. (*Evidence: notebook part 5.*)

Is the evaluation methodology appropriate? Partly. the pipeline mechanics are correct, but reporting accuracy on a 0.17%-positive problem and omitting PR-AUC/MCC is not. Other limitations: a single train/test split (no cross-validation for the headline numbers), no threshold tuning, no error analysis, and no discussion of the FP/FN cost asymmetry.

Are the conclusions justified? The qualitative conclusion (ML can help detect fraud) is reasonable, but the **quantitative claims are not supported by evidence**. Our reproduction contradicts the “near-perfect” framing, and we state this explicitly.

Table 2: Faithful reproduction of the author’s pipeline on the untouched, imbalanced test set (20% split, train-only SMOTE). The claimed “LogReg: 100% recall / 94% precision” is not observed.

Model (author repro)	Accuracy	Precision	Recall	F_1	MCC	PR-AUC
Logistic Regression	0.9737	0.0530	0.8737	0.100	0.212	0.675
Random Forest	0.9995	0.9211	0.7368	0.819	0.824	0.803
<i>Claimed (README)</i>	<i>0.998</i>	<i>0.94</i>	<i>1.00</i>	—	—	—

4 Feature Engineering Analysis

- **Was feature engineering performed?** In the source, minimally (scaling). We add $\log(1 + \text{Amount})$ and `hour_of_day`.
- **Features used.** V1–V28 (PCA components), `Time`, `Amount`, engineered `log_Amount` and `hour_of_day`.
- **Transformations and why:**
 - $\log(1 + \text{Amount})$: `Amount` is heavily right skewed (skew ≈ 16.98). The log compresses the tail so scale-sensitive models are not dominated by a few large transactions.
 - **StandardScaler** on `Time`/`Amount`/engineered columns: puts them on the same scale as the already $\sim$ standardized PCA components. Fit on **train only** to avoid leakage.
 - `hour_of_day` from `Time`: Exposes temporal structure in the fraud rate.
- **Redundancy.** V1–V28 are orthogonal by PCA construction, so linear redundancy is low by design; `Amount` and `log_Amount` are monotonically related. We detect redundancy via a correlation heatmap and drop or combine features.

- **Additional useful features:** Information such as how often a card is used in a short time period, the time since the previous transaction, and whether the spending amount is unusual compared to the customer’s normal behavior could improve detection. These features are not available in this dataset because the original data was anonymized.

5 Reproducibility Analysis

- **Does the code run? Yes.** The source code, and my notebook run successfully using the public dataset.
- **Files and dependencies available?** Dataset is public, the CSV is not committed (size), versions are pinned via `requirements.txt`.
- **Hidden preprocessing** The PCA transform (V1–V28) is applied by the data provider and cannot be reproduced from raw features.
- **Overall reproducibility.** Good at the code level (fixed seeds, correct pipeline order), but the README’s reported numbers are **not reproducible** from the code on the real data.

6 Experimental Results

- **Experiments.** Faithful reproduction of the author’s pipeline. An improved pipeline with five models (Dummy, Logistic Regression, Random Forest, XGBoost, Isolation Forest). A SMOTE-in-CV honest variant, a leakage counterfactual, and PR-curve threshold tuning.
- **Modifications.** Engineered features (`log_Amount`, `hour_of_day`), imbalance-aware metrics (PR-AUC/MCC/ F_2), XGBoost and Isolation Forest models, PR-curve threshold selection, and error analysis - all absent from the source.
- **Models trained and key hyper-parameters.**
 - **Dummy** - `strategy="most_frequent"` (predicts “legit” always). The accuracy-fallacy baseline.
 - **Logistic Regression** — `max_iter=1000`, `class_weight="balanced"`.
 - **Random Forest** — 200 trees, `class_weight="balanced"`.
 - **XGBoost** — 300 trees, `max_depth=4`, `learning_rate=0.1`, `subsample=0.9`, `scale_pos_weight=599` (the negative/positive ratio, i.e., cost-sensitive learning), `eval_metric="aucpr"`.
 - **Isolation Forest** — 200 estimators, `contamination` set to the training fraud rate, *unsupervised* anomaly detection.
- **Metrics.** Accuracy, Precision, Recall, F_1 , F_2 , MCC, ROC-AUC, PR-AUC, confusion matrix.

Results. Table 3.

Table 3: Consolidated model comparison on the test set. Best honest supervised model by PR-AUC: XGBoost. All models predict at the default 0.5 threshold here (tuning is applied in part 6 in the notebook).

Model	Acc.	Prec.	Recall	F_1	F_2	MCC	PR-AUC
XGBoost (cost-sensitive)	0.9995	0.8810	0.7817	0.828	0.800	0.830	0.824
Random Forest (class-weighted)	0.9995	0.9381	0.7465	0.831	0.778	0.837	0.817
LogReg + SMOTE-in-CV (honest)	0.9721	0.0519	0.9094	0.098	0.211	0.213	0.741
Logistic Regression (weighted)	0.9729	0.0521	0.8873	0.098	0.211	0.211	0.698
Isolation Forest (unsupervised)	0.9972	0.1655	0.1620	0.164	0.163	0.162	0.087
Dummy (always legit)	0.9983	0.0000	0.0000	0.000	0.000	0.000	0.002

Reading the table. The **Dummy** row is the punchline: the *highest* accuracy (99.83%) belongs to the model that detects *nothing*. Ranking instead by PR-AUC / MCC / F_2 puts the **tree ensembles on top**: XGBoost (PR-AUC 0.824) and Random Forest (MCC 0.837) dominate. Logistic Regression achieves the highest recall (88.7%) but at a catastrophic precision of 5.2% - for every true fraud it flags, it raises ~ 18 false alarms.

The unsupervised Isolation Forest is the weakest (PR-AUC 0.087): without labels it cannot learn the subtle multivariate fraud boundary.

Conclusion: contrary to the README, Logistic Regression is *not* the best model, and even the model’s that gave the best results are not near perfect as stated in the source.

7 Error Analysis

Threshold tuning first. The default 0.5 threshold is arbitrary. We select an operating point from the precision-recall curve of the best model (**XGBoost**, PR-AUC = 0.824), targeting a business goal of **recall** ≥ 0.85 and taking the highest-precision threshold that meets it: $t^* = 0.066$. At t^* the confusion matrix on the test set is:

Table 4: XGBoost confusion matrix at the tuned threshold $t^* = 0.066$ (target recall ≥ 0.85).

	Predicted legit	Predicted fraud
Actual legit	$TN = 84,898$	$FP = 78$
Actual fraud	$FN = 21$	$TP = 121$

This yields **recall** = 85.2%, **precision** = 60.8%, $F_2 = 0.79$: we catch 121 of 142 frauds while raising only 78 false alarms out of 84,976 legitimate transactions.

Failure examples and patterns. Among the 21 fraud cases that the model missed (false negatives), the median standardized **Amount** was -0.36 , compared with -0.29 for the 121 fraud cases that were correctly detected. This suggests that **missed fraud transactions are usually smaller and look more like normal transactions**, making them harder for the model to identify. The 78 false positives were legitimate transactions that had feature patterns similar to fraudulent transactions.

FP/FN trade-off. A false negative (FN) means that a fraud transaction was missed, which can cause direct financial loss and reduce customer trust. A false positive (FP) means that a normal transaction was incorrectly flagged as fraud, which can block customers and increase support costs. Since missing fraud is usually more costly, we lower the decision threshold to detect more fraud cases, even if this creates more false alarms. The notebook’s threshold analysis shows that FN and FP change in opposite directions as the threshold changes, demonstrating the trade-off and supporting the selected threshold t^* .

Cost-based decision. Choosing the best threshold depends on the cost of mistakes. Missing a fraud transaction usually costs more than incorrectly blocking a normal transaction. Therefore, the model should balance false negatives (FN) and false positives (FP) based on their real-world costs. At the selected threshold t^* , the model detects most fraud cases while creating some false alarms. However, the source does not discuss this trade-off and only reports results using a fixed threshold of 0.5, which does not consider different business needs.

8 Conclusions

- **Main findings.** Accuracy is not a good measure for this problem because fraud cases are rare. The author’s reported “100% recall / 94% precision” results could not be reproduced on the test set. The model performs well, but the results are not perfect.
- **Lessons learned.** It is important to use suitable evaluation metrics such as PR-AUC, MCC, and F_2 -score instead of relying only on accuracy. A single good result should not be presented as a general performance measure. It is important to optimize the threshold according to our preferences.
- **Strengths of the source.** The project is clear, easy to run, and well organized. It correctly applies data splitting, scaling, SMOTE, and model evaluation.
- **Weaknesses.** The source focuses too much on accuracy and reports results that are difficult to reproduce. It does not include enough analysis of different thresholds, errors, or additional evaluation metrics.
- **Future improvements.** Possible improvements include using cost-sensitive models, stronger algorithms such as gradient boosting, adding more useful transaction features, improving probability estimates, and choosing a decision threshold based on the precision-recall trade-off.